

BITÁCORA

Taller 3: Simplificación de Heightmap por Vecindad de Orden k

Abel Albuez · Santiago Gil Gallego

Geometría Computacional — Pontificia Universidad Javeriana
Maestría en Ingeniería de Sistemas y Computación — Bogotá, 2026
Profesor: Leonardo Flórez-Valencia

1. El punto de partida

El código base del profesor entregaba ya una parte del trabajo resuelta: dada una imagen PNG de heightmap, el programa leía cada píxel como un punto en el plano y construía sobre él una triangulación de Delaunay completa. Una imagen de 512×512 produce 262.144 vértices y más del doble de triángulos. La estructura era exactamente la face-vertex mesh de la Clase 6. Cada vértice tenía su posición (x, y) y su altura z en `v->info()`, normalizada a $[0, 1]$ por `pujCGAL::Heightmap`.

Nuestro trabajo comenzaba entre la línea 42 y la 44: después de asignar las alturas y antes de guardar en disco. En ese espacio había que meter un algoritmo que decidiera cuáles vértices sobran y cuáles importan.

2. El problema geométrico: ¿cuándo sobra un vértice?

Si todos los vecinos de p tienen una altura muy similar a la suya, la zona es plana y p es redundante. Si algún vecino tiene una altura muy distinta, p está en un borde o cresta. Esta intuición se formaliza con el error de planitud:

$\text{error}(p) = \text{promedio de } |p.z - q_i.z| \text{ para cada vecino } q_i \text{ de } p$

Si $\text{error}(p) < \epsilon \rightarrow$ zona plana $\rightarrow$ eliminar. Si $\text{error}(p) \geq \epsilon \rightarrow$ borde o cresta $\rightarrow$ conservar.

Criterio de Planitud: ¿cuando sobra un vertice?

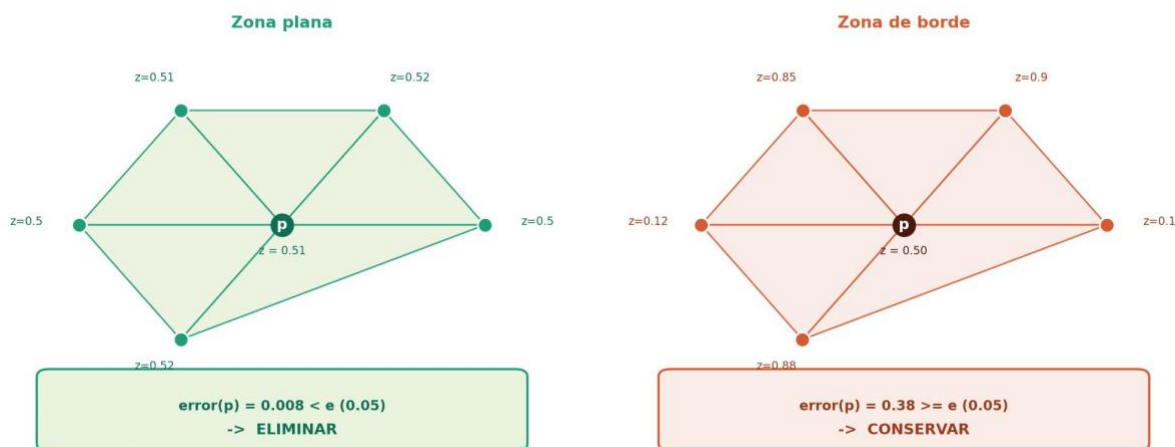


Figura 1 — Criterio de planitud: zona plana (error bajo $\rightarrow$ eliminar) vs borde (error alto $\rightarrow$ conservar).

Por qué este criterio y no otro

Evaluamos tres alternativas: varianza de alturas (penaliza diferencias entre vecinos, no respecto a p — descartada), distancia al plano por mínimos cuadrados ($O(\text{grado}^2)$ — costoso), y promedio de diferencias absolutas ($O(\text{grado})$, simétrico, escala con $[0, 1]$ — elegido).

Calibración de epsilon

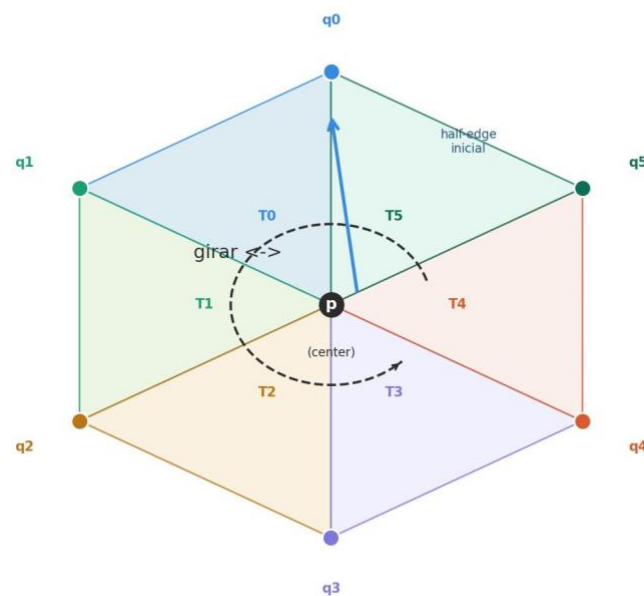
ϵ	Vértices después	Reducción	Observación
0.01	15,133	90.9%	Conservador
0.03	14,376	91.4%	Equilibrado
0.05	13,830	91.7%	★ Recomendado para terreno real
0.08	12,907	92.2%	Agresivo
0.12	11,865	92.9%	Muy agresivo

3. Cómo se identifican los vecinos: la DCEL en acción

El `Face_circulator` de CGAL implementa el recorrido `twin→next` de la DCEL de la Clase 6. Girar ese circulator alrededor de `p` visita todos los triángulos de su estrella en orden circular. De cada triángulo se extraen los vértices que no son `p`.

```
TFaceCirculator fc = T.incident_faces(p), done = fc;
do {
    if(!T.is_infinite(fc)) {
        for(int i = 0; i < 3; i++) {
            TVertexHandle q = fc->vertex(i);
            if(q != p && !T.is_infinite(q))
                vecinos.insert(q); // std::set elimina duplicados
        }
    }
} while(++fc != done);
```

**Face_circulator: Recorrido de la Estrella de p
(DCEL half-edge twin->next)**



Face_circulator recorre $T0 \rightarrow T1 \rightarrow \dots \rightarrow T5 \rightarrow T0$

Cada cara tiene 3 vértices: $p + 2$ vecinos $\rightarrow$ `std::set` elimina duplicados

Figura 2 — `Face_circulator` girando alrededor de `p`. Cada cara $T0$ - $T5$ aporta dos vecinos; el `std::set` los filtra automáticamente.

Por qué `std::set` y no `vector`

Cada vecino aparece en exactamente dos triángulos adyacentes. Un `vector` guardaría cada vecino dos veces. El `std::set` los elimina en inserción en $O(\log n)$ — más limpio que cualquier deduplicación manual.

Expansión a orden k

Vecindad de Orden k - Expansion por Anillos

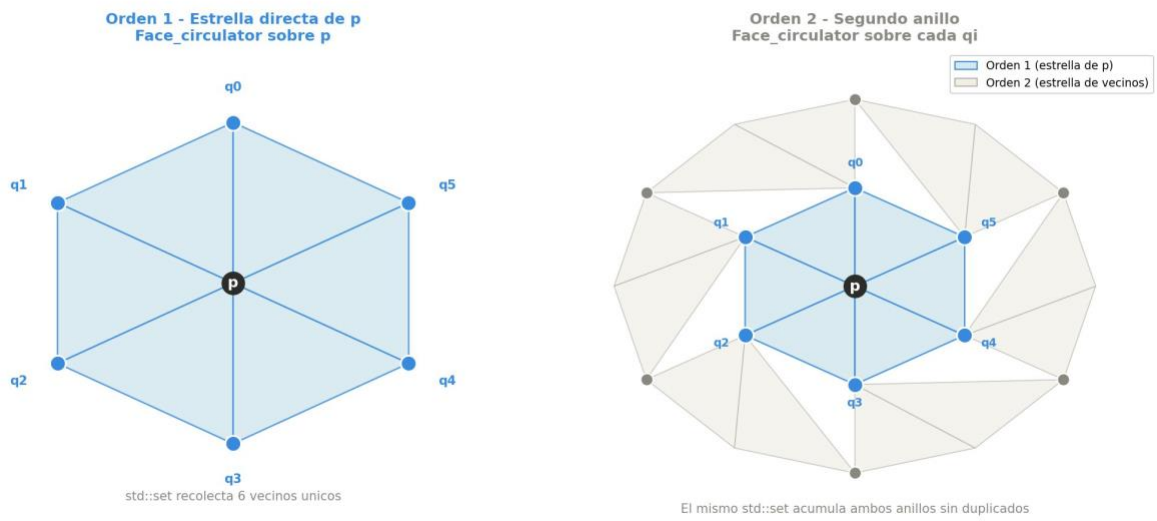


Figura 3 — Vecindad de orden 1 (estrella azul de p) y orden 2 (segundo anillo gris). El mismo `std::set` acumula ambos sin duplicados.

Para orden 2, repetimos el mismo circulator desde cada vecino q_i del orden 1. Los triángulos nuevos forman el segundo anillo y se acumulan en el mismo set.

4. El orden de procesamiento: Dijkstra

El profesor mencionó Dijkstra en clase. La conexión: cola de prioridad (min-heap) que siempre procesa el nodo de menor costo. Aquí el costo es el error de planitud — el vértice más plano sale primero. El orden importa porque eliminar p cambia el entorno de sus vecinos; procesar primero el más prescindible maximiza la simplificación progresiva de adentro hacia afuera.

priority_queue - Menor error de planitud sale primero
(Dijkstra: siempre procesar el mas prescindible)

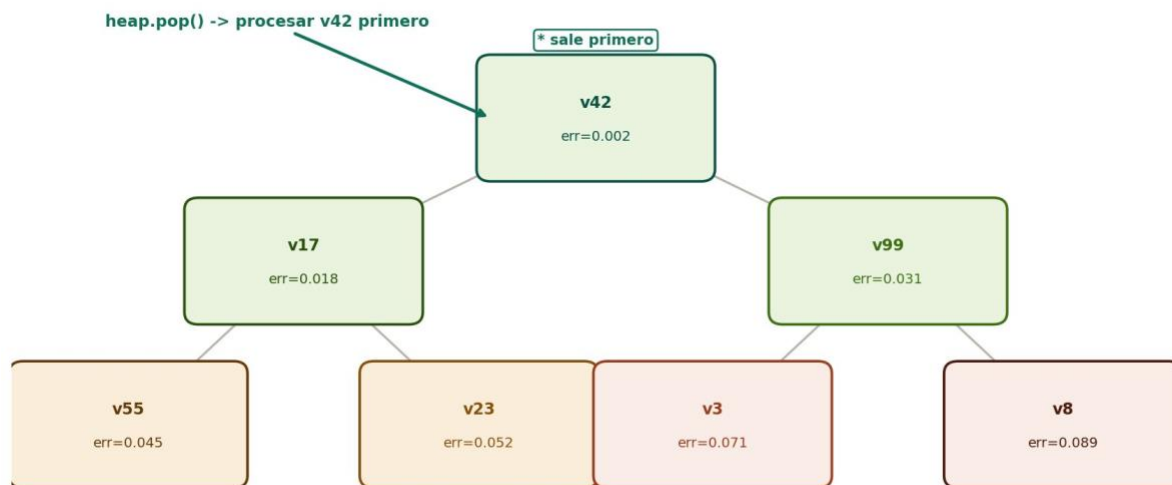


Figura 4 — Min-heap ordenado por error de planitud. v_{42} (err=0.002) sale primero. Degradé verde→rojo indica urgencia de eliminación.

Lazy deletion: por qué y cómo

Cuando eliminamos p , sus vecinos se reinsertan con nuevo error. La versión vieja sigue en el heap. No hay decrease-key en la STL, así que usamos lazy deletion: dos chequeos descartan entradas obsoletas al salir. No fue visto en clase, pero emerge naturalmente al usar priority_queue de la STL.

Lazy Deletion: detección de entradas obsoletas en el heap

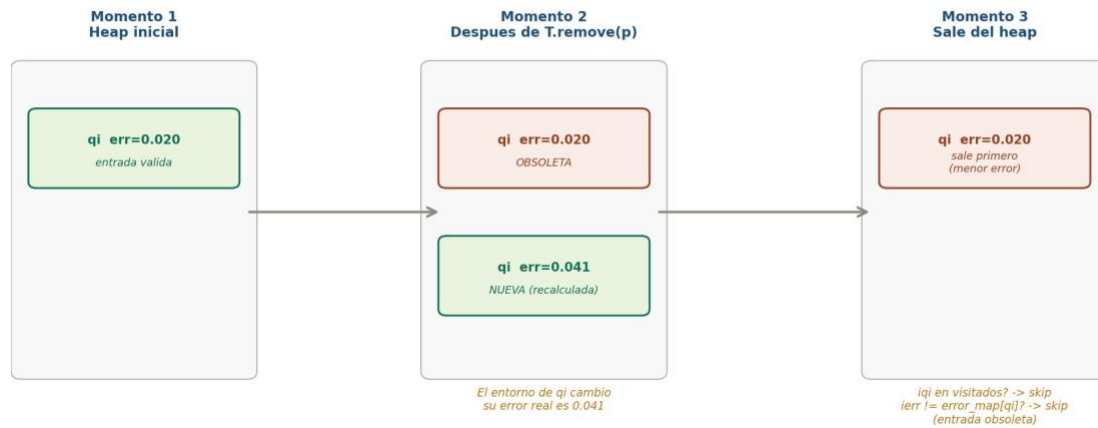


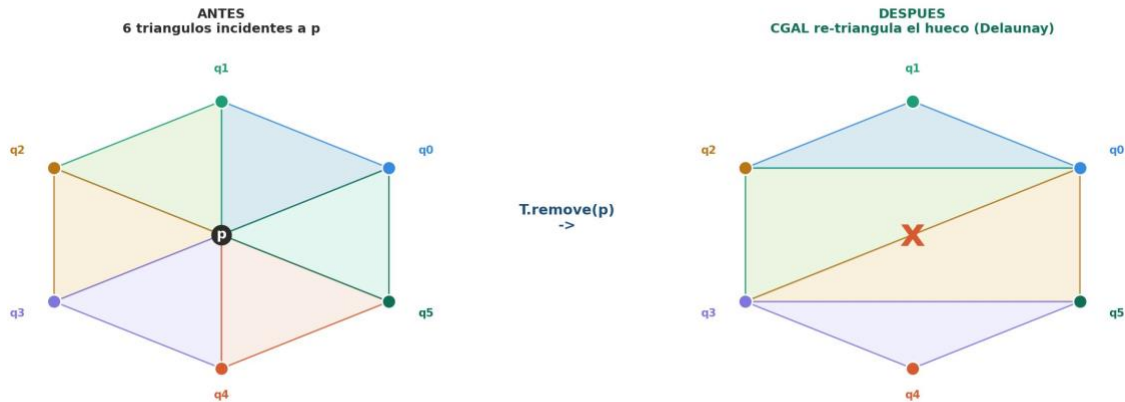
Figura 5 — Los 3 momentos del heap: entrada válida → obsoleta tras T.remove(p) → sale y se descarta con los dos chequeos.

- unordered_set visitados — $O(1)$: si el vértice ya fue procesado → skip.
- unordered_map error_map — $O(1)$: si el error del heap no coincide con el mapa → entrada obsoleta → skip.

5. Qué pasa cuando se elimina un vértice

Cuando el error de p es menor que ϵ , llamamos $T.remove(p)$. CGAL elimina todos los triángulos de la estrella de p y re-triangua el hueco con el criterio de Delaunay. La malla sigue siendo válida y Delaunay — sin intervención adicional.

Efecto de $T.remove(p)$ en la Triangulación



La malla sigue siendo Delaunay - CGAL garantiza validez automáticamente

Figura 6 — $T.remove(p)$: los 6 triángulos de la estrella desaparecen. CGAL conecta los vecinos directamente con Delaunay.

5.1 Muestra de ejecución: 3 ciclos

Los tres ciclos siguientes ilustran el comportamiento con $\epsilon = 0.05$ sobre un heightmap real:

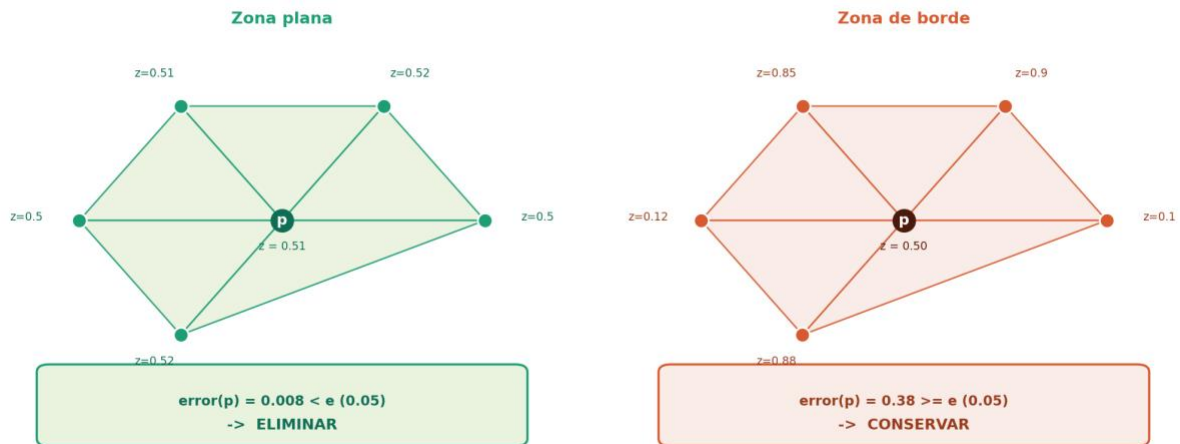
Ciclo	Detalle
Ciclo 1	$p: v_{42} \rightarrow \text{error} = 0.002$ Vecinos: $q_0 \dots q_5$ con Face_circulator (orden 1) $0.002 < 0.05 \rightarrow \text{ELIMINAR}$ $T.remove(v_{42})$. Vecinos recalculan error y reinseran en heap.
Ciclo 2	$p: q_1 \rightarrow \text{error} = 0.008$ (recalculado - v_{42} ya no existe) Vecinos: estrella de q_1 sin v_{42} $0.008 < 0.05 \rightarrow \text{ELIMINAR}$ $T.remove(q_1)$. La zona plana continúa simplificándose.
Ciclo 3	$p: v_{\text{cresta}} \rightarrow \text{error} = 0.12$ Vecinos: estrella del borde — triángulos de pendiente pronunciada $0.12 \geq 0.05 \rightarrow \text{CONSERVAR}$ v_{cresta} permanece. Loop continúa con el siguiente del heap.

Los primeros ciclos eliminan vértices de zona plana en rápida sucesión. Cuando el heap entrega un vértice de borde (Ciclo 3), el loop lo conserva y continúa.

5.2 Visualización paso a paso (primer frame del GIF)

El script visualizer.py genera un GIF de 7 pasos para cada imagen procesada. El primer frame del GIF correspondiente a NormalMap.png ($262.144 \rightarrow 1.647$ vértices, 99.4% de reducción):

Criterio de Planitud: ¿cuando sobra un vertice?



(1/7) Paso 1: Criterio de planitud
¿cuando sobra un vertice?

Figura 7 — Frame 1/7 del GIF paso_a_paso.gif — NormalMap.png (262.144 vértices). El GIF completo está en output/test-NormalMap/paso_a_paso.gif.

6. Estructuras de datos: por qué cada una

Estructura	Concepto / por qué esa	Justificación
Face_circulator	DCEL — Clase 6	Recorre la estrella de p con $\text{twin} \rightarrow \text{next}$ sin exponer punteros.
std::set (vecinos)	Árbol rojo-negro	Elimina duplicados al insertar. Cada vecino aparece en dos caras — el set lo guarda una sola vez.
priority_queue + greater<>	Dijkstra	Min-heap: el vértice con menor error sale primero.
unordered_set (visitados)	Tabla hash $O(1)$	Marca vértices ya procesados. $O(1)$ vs $O(\log n)$ del set — se llama miles de veces.
unordered_map (error_map)	Tabla hash $O(1)$	Guarda el error más reciente. Detecta entradas obsoletas del heap en $O(1)$.

La distinción entre `unordered_set/map` y sus versiones ordenadas fue deliberada. En imágenes de 2 millones de vértices, la diferencia entre $O(1)$ y $O(\log n)$ por búsqueda se acumula en decenas de millones de operaciones.

7. Bugs encontrados y por qué importan

Bug	Síntoma y causa	Solución
IO::save sobreescribe v->info()	IO::save asigna índices OBJ en v->info() al generar el archivo. El algoritmo calculaba errores sobre índices — reducción 0%.	Backup de alturas en unordered_map antes del save. Restaurar inmediatamente después.
break agresivo en el heap	Entradas obsoletas con error alto detenían el loop antes de procesar vértices válidos con menor error.	Cambiar break por continue. El loop sigue hasta vaciar el heap completamente.
Vértice central fuera de la malla	vo[len//2] no es el vértice geoméricamente central — CGAL inserta en orden interno de Delaunay.	Calcular centroide geométrico y buscar el vértice más cercano con argmin de distancia.

El bug del IO::save fue el más inesperado. La reducción era 0% — el algoritmo no parecía hacer nada. IO::save() asigna índices correlativos en v->info() al generar el .obj. Efecto secundario no documentado en pujCGAL. La lección: siempre verificar qué campos modifica la biblioteca antes de asumir que son estables.

8. Resultados

Ejecutamos con $\varepsilon = 0.05$ y $\text{orden_k} = 2$ sobre 6 imágenes. Las SRTM_US son datos reales del satélite SRTM. Las NormalMap son heightmaps sintéticos con espiral concéntrica.

Imagen	Tipo	Antes	Después	Reducción
SRTM_US_256.png	Terreno EE.UU. 256px	32,768	4,840	85.2%
SRTM_US_512.png	Terreno EE.UU. 512px	131,072	14,190	89.2%
NormalMap-second.png	Normal map espiral	196,608	22,352	88.6%
NormalMap.png	Normal map completo	262,144	1,647	99.4%
SRTM_US_1024.png	Terreno EE.UU. 1024px	524,288	37,188	92.9%
SRTM_US_2048.png	Terreno EE.UU. 2048px	2,097,152	81,758	96.1%

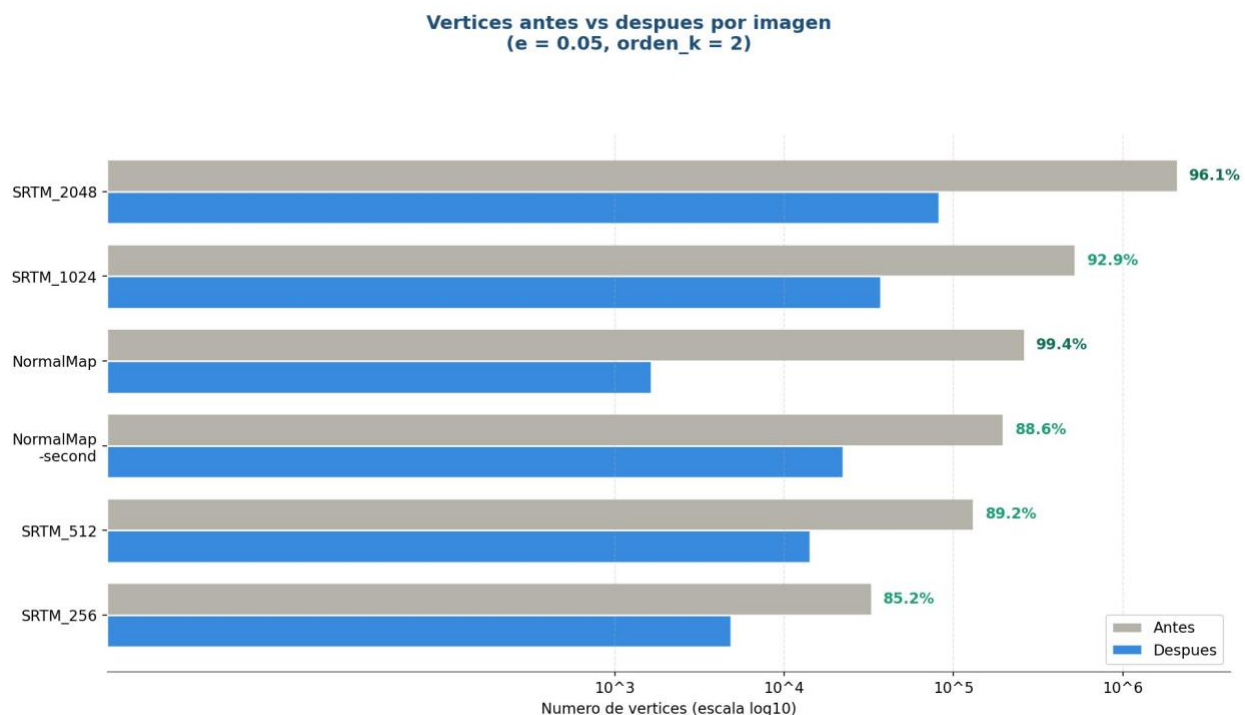


Figura 8 — Vértices antes vs después en escala logarítmica ($\varepsilon=0.05$, $k=2$). El % de reducción aparece al final de cada barra.

- La reducción escala con el tamaño. Imágenes más grandes tienen proporcionalmente más vértices en zonas planas.
- NormalMap.png alcanza 99.4% — sus zonas de gradiente uniforme tienen error ≈ 0 . Solo los bordes de cada anillo sobreviven.

9. La solución del profesor: por qué genera tantos triángulos en zonas homogéneas

El profesor entregó también una solución alternativa al taller, con un pipeline distinto al nuestro: en vez de decimar directamente sobre la triangulación de Delaunay con el criterio de error promedio, parte el problema en dos ejecutables encadenados. Primero construye el Delaunay completo desde el heightmap (`build_delaunay_from_heightmap.cxx`), y después sobre esa malla aplica un criterio estadístico para decidir qué vértices sobreviven (`decimate_mesh.cxx`). Replicar y estudiar esa solución nos abrió una pregunta que el profe nos dejó explícita: por qué en zonas homogéneas se siguen generando tantos triángulos.

9.1 El criterio del profesor

Para cada vértice p , el profe calcula con un BFS por anillos (método de Welford) la media μ y la varianza σ^2 de las alturas de su vecindad de orden k . Después aplica un test relativo: si $|h(p) - \mu|$ es menor que $\gamma \cdot \sigma$, el punto se considera redundante y se descarta; en caso contrario se conserva. Aquí γ actúa como un multiplicador de la desviación estándar local, una especie de tolerancia relativa al ruido del propio vecindario.

9.2 Lo que vimos en la práctica

Corrimos el pipeline del profe sobre `SRTM_US_scaled_512` ($512 \times 256 = 131.072$ vértices) con $\gamma=1.0$ y $\text{order}=2$. La reducción fue de apenas 47%, dejando 69.170 vértices sobrevivientes. Pero lo más revelador no fue el porcentaje sino la distribución: en el panel de la malla simplificada, el océano y las llanuras conservaban casi tanta densidad como las Rocosas y los Apalaches. Las zonas homogéneas, que deberían haberse decimado más agresivamente, tenían triangulaciones similares a las zonas accidentadas. Eso no era ruido; era el algoritmo comportándose al revés de lo que la intuición geométrica sugiere.

9.3 El diagnóstico: el caso patológico de $\sigma \rightarrow 0$

El test $|h(p) - \mu|$ menor que $\gamma \cdot \sigma$ es relativo a la varianza local. Eso suena razonable, pero falla en los extremos. Comparemos dos vecindarios con un ejemplo numérico de siete píxeles cada uno:

Magnitud	Vecindario plano (océano)	Vecindario accidentado (cresta)
Alturas h	[0, 0, 0, 0, 0, 0, 0]	[10, 50, 80, 100, 90, 30, 20]
Media μ	0	~54
Desviación σ	0	~35
Banda $\gamma \cdot \sigma$ (con $\gamma=1$)	0	~35
Test $ h-\mu $ menor que $\gamma \cdot \sigma$	0 menor que 0 → FALSO	depende del punto, frecuente

El caso del océano es la trampa. En una zona perfectamente plana, σ colapsa a cero porque todas las alturas son idénticas. La banda de tolerancia $\gamma \cdot \sigma$ también es cero. Y el test 0 menor que 0 es falso, lo que significa que el algoritmo concluye que el punto no es plano y lo conserva. El criterio del profe deja de detectar planitud justo donde la planitud es absoluta. Y como el SRTM tiene grandes regiones uniformes (océano, llanuras), la mayor parte de los puntos sobreviven sin ser cuestionados.

Por eso vemos tantos triángulos en zonas homogéneas: no es que el algoritmo decida conservarlos por una buena razón geométrica; es que el test relativo no aplica cuando la

varianza local es nula. El criterio mide qué tan lejos está un punto de la media de su vecindario en unidades de la propia variabilidad del vecindario, y si esa variabilidad no existe, no hay cómo medir la distancia.

9.4 La propuesta: medir planitud directamente con σ

El cambio que proponemos es conceptualmente mínimo: dejar de comparar $|h(p) - \mu|$ contra $\gamma \cdot \sigma$ y comparar σ directamente contra un umbral. La nueva regla queda:

si σ menor que $\gamma \rightarrow$ vecindario plano $\rightarrow p$ mata a sus vecinos

El parámetro γ cambia de naturaleza: ya no es un multiplicador de la desviación estándar, sino un umbral absoluto en las mismas unidades de la altura. Un γ pequeño (digamos 5 unidades de elevación) significa que solo zonas casi perfectamente planas se decimann; un γ más grande (digamos 30) considera planas también las laderas suaves y permite más decimación.

Volviendo a los dos vecindarios del ejemplo: con la regla nueva, el océano ($\sigma=0$) cumple σ menor que γ para cualquier γ positivo y se decimann sus vecinos. La cresta ($\sigma \sim 35$) no cumple σ menor que 5 ni siquiera con $\gamma=10$, así que se preserva. El comportamiento se invierte respecto al criterio original justo en el lugar correcto: ahora sí se decimann las zonas homogéneas y se preservan las accidentadas.

Geométricamente esto tiene sentido: una zona plana es por definición una zona donde la variabilidad de las alturas es baja, y σ mide exactamente eso. El criterio original tenía sentido como detector de outliers dentro de una distribución con varianza no nula, pero un heightmap real no es una distribución estadística: tiene regiones enteras donde la varianza local es genuinamente cero, y ahí el test pierde significado.

10. Conclusiones

La DCEL no es solo teoría

El `Face_circulator` es exactamente el recorrido `twin→next` de la DCEL de la Clase 6. Entender la teoría permitió implementar el circulator directamente, sin ensayo y error.

Dijkstra es una estrategia, no solo un algoritmo de rutas

La misma cola de prioridad que resuelve rutas más cortas ordena aquí vértices por planitud. La lazy deletion emerge naturalmente al usar `priority_queue` de la STL sin `decrease-key` — no fue vista en clase pero es su consecuencia directa.

CGAL: precisión con efectos secundarios

`T.remove()` garantiza validez y Delaunay automáticamente. Pero `IO::save()` modifica `v->info()` silenciosamente. Cuando un resultado es inesperadamente incorrecto, el primer paso es verificar qué campo modificó la biblioteca.

Un criterio relativo no siempre dice lo que parece decir

Estudiar la solución del profesor nos dejó un aprendizaje que no esperábamos: el criterio $|h(p) - \mu|$ menor que $\gamma \cdot \sigma$, que parece una elegante medida de planitud relativa, falla justo donde la planitud es más clara — cuando σ colapsa a cero. Un test relativo necesita que el denominador implícito tenga sentido, y en regiones genuinamente homogéneas no lo tiene. La lección es geométrica: la planitud no se mide bien con una distancia normalizada por una variabilidad que puede ser nula; se mide directamente con esa variabilidad. Cuando un criterio estadístico se comporta al revés de lo que la intuición dicta, vale la pena revisar si está midiendo lo que uno cree que está midiendo.

Repositorio: <https://github.com/AbelAlbuez/computational-geometry-cgal>